

Task –1 To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

```

GNU nano 2.7.1 variable_expansion.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 1000

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            i++;

            char variable[100];
            int k = 0;

            while (isalnum(input[i]) || input[i] == '_')
            {
                variable[k++] = input[i++];
            }

            variable[k] = '\0';

            if (k == 0)
            {
                output[j++] = '$';
                continue;
            }

            char *value = getenv(variable);

            if (value != NULL)
            {
                strcpy(&output[j], value);
                j += strlen(value);
            }
            else
            {
                printf("Undefined variable: %s\n", variable);
            }
        }
    }

    output[j] = '\0';
}

int main()
{
    char input[MAX];
    char output[MAX];

    printf("Variable Expansion Program\n");
    printf("Enter text using $VARIABLE:\n");

    fgets(input, sizeof(input), stdin);

```

```

        {
            printf("Undefined variable: %s\n", variable);
        }
    }
    else
    {
        output[j++] = input[i++];
    }
}

output[j] = '\0';
}

int main()
{
    char input[MAX];
    char output[MAX];

    printf("Variable Expansion Program\n");
    printf("Enter text using $VARIABLE:\n");

    fgets(input, sizeof(input), stdin);

```

```

gopinath@GOPINATH:~/practical_memory$ gcc variable_expansion.c -o variable_expansion
gopinath@GOPINATH:~/practical_memory$ ./variable_expansion
Variable Expansion Program
Enter text using $VARIABLE:
Hello $USER

Input: Hello $USER
Expanded: Hello gopinath
gopinath@GOPINATH:~/practical_memory$ ./variable_expansion
Variable Expansion Program
Enter text using $VARIABLE:
Home: $HOME, Path: $PATH

Input: Home: $HOME, Path: $PATH
Expanded: Home: /home/gopinath, Path: /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/WINDOWS/system32:/mnt/c/Win
DOWS:/mnt/c/WINDOWS/system32/WindowsPowerShell/v1.0:/mnt/c/KI
NDOWS/System32/OpenSSH;/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/U
sers/gopinath/AppData/Local/Programs/Python/Python313;/mnt/c/Users/gopinath/AppData/
Local/Programs/Python/Python313;/mnt/c/Users/gopinath/AppData/Local/Microsoft/WindowsApps;
/mnt/c/Users/gopinath/AppData/Local/Programs/Microsoft VS Code/bin;/mnt/c/Program Files/
MySQL/MySQL Server 8.0/bin;/mnt/c/Users/gopinath/AppData/Local/Microsoft/WindowsApps:/mnt/c/
Users/gopinath/AppData/Local/Programs/Microsoft VS Code/

Command not found
Executable: command not found

```

Task 2 - To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```

#define MAX_INPUT 100

// Function declarations
void shell_pwd(char *args[]);
void shell_cd(char *args[]);
void shell_help(char *args[]);
void shell_exit(char *args[]);

// Built-in command structure
struct builtin
{
    char *name;
    void (*function)(char *args[]);
};

// Dispatch table
struct builtin builtins[] =
{
    {"pwd", shell_pwd},
    {"cd", shell_cd},
    {"help", shell_help},
    {"exit", shell_exit}
};

int builtin_count = 4;

// pwd command
void shell_pwd(char *args[])
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

// cd command
void shell_cd(char *args[])
{
    if (args[1] == NULL)
    {
        printf("cd: missing directory\n");
        return;
    }
}

```

```

    }

    if (chdir(args[1]) != 0)
        perror("cd");
}

// help command
void shell_help(char *args[])
{
    printf("Built-in commands:\n");
    printf("  pwd - Display current directory\n");
    printf("  cd  - Change directory\n");
    printf("  help - Display available commands\n");
    printf("  exit - Exit the shell\n");
}

// exit command
void shell_exit(char *args[])
{
    printf("Exiting shell...\n");
    exit(0);
}

int main()
{
    char input[MAX_INPUT];
    char *args[10];

    printf("Mini Shell Started\n");
    printf("Type 'help' to see available commands.\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        int count = 0;

```

```

        char *token = strtok(input, " ");
        while (token != NULL && count < 9)
        {
            args[count++] = token;
            token = strtok(NULL, " ");
        }

        args[count] = NULL;
        int found = 0;

        // Search dispatch table
        for (int i = 0; i < builtin_count; i++)
        {
            if (strcmp(args[0], builtins[i].name) == 0)
            {
                builtins[i].function(args);
                found = 1;
                break;
            }
        }

        if (!found)
        {
            printf("Invalid command: %s\n", args[0]);
        }

        return 0;
    }
}

```

```
gopinath@GOPINATH:~/practical_memory$ gcc variable_expansion.c -o variable_expansion
gopinath@GOPINATH:~/practical_memory$ ./variable_expansion
Variable Expansion Program
Enter text using $VARIABLE:
Hello $USER

Input: Hello $USER
Expanded: Hello gopinath

Enter text using $VARIABLE:
Home: $HOME, Path: $PATH

Input: Home: $HOME, Path: $PATH
Expanded: Home: /home/gopinath, Path: /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin

Enter text using $VARIABLE:
$MYHOST

Input: $MYHOST
Expanded:

Enter text using $VARIABLE:
Invalid $VAR

Input: Invalid $VAR
Expanded: Invalid $VAR

Variable expansion completed.
```